

COMICCRAFT

AI Comic Story Creator using Gemini Models

PROJECT REPORT

Structured in the Required 11-Chapter Format

Submitted by: _____

Register Number: _____

Department / Institution: _____

Academic Year: _____

Certificate

This is to certify that the project report entitled “ComicCraft – AI Comic Story Creator using Gemini Models” is submitted as part of the project requirements. The report describes the design, implementation approach, testing areas, and results of the web-based application.

Project Guide: _____

Head of Department: _____

Date: _____ Place: _____

(Signature and seal)

Declaration

I hereby declare that this project report titled “ComicCraft – AI Comic Story Creator using Gemini Models” is prepared for academic submission. The project description and technical details are based on the supplied project documentation and the application screenshots included in this report.

Student Name: _____

Signature: _____

Acknowledgement

I express my sincere gratitude to the institution, department, project guide, and all those who supported the preparation of this project. I also acknowledge the documentation and software resources related to FastAPI, Python, Gemini, Stable Diffusion, Jinja2, and PDF generation that informed the project.

Abstract

ComicCraft is a web-based Generative AI application that transforms a user's creative prompt into a five-panel comic story. The user supplies a story prompt, character name, setting, tone, and art style. FastAPI coordinates the request, Gemini models create a structured outline and detailed narration/dialogue, and Stable Diffusion generates illustrations. A Jinja2 interface displays the comic panel by panel, while an export component creates a downloadable PDF using FPDF.

The report presents the project description, Generative AI platform overview, analysis of existing and proposed workflows, architecture, environment requirements, six implementation milestones, testing and validation, results, challenges, future enhancements, conclusion, references, and appendix.

Screenshots supplied with the project are placed in the relevant chapters as evidence of the interface and observed output.

Table of Contents

Chapter 1 – Introduction

1.1 Project Description

1.2 Problem Statement

1.3 Objectives

Chapter 2 – Generative AI Platform Overview

2.1 Introduction to Generative AI

Chapter 3 – Project Analysis

3.1 Existing Behaviour

3.2 Problems in Existing Behaviour

3.3 Proposed Solution

3.4 Project Scenarios / Use Cases

Chapter 4 – System Architecture and Design

4.1 Architecture Overview

4.2 Project Workflow

Chapter 5 – Requirements and Environment Setup

5.1 Hardware Requirements

5.2 Software Requirements

5.3 ServiceNow Instance Requirements

Chapter 6 – Project Implementation

6.1 Milestone 1

6.2 Milestone 2

6.3 Milestone 3

6.4 Milestone 4

6.5 Milestone 5

6.6 Milestone 6

Chapter 7 – Testing and Validation

Chapter 8 – Results and Discussion

Chapter 9 – Challenges and Troubleshooting

Chapter 10 – Future Enhancements

Chapter 11 – Conclusion

References

List of Figures

Figure 8.1 ComicCraft input form in browser

Figure 8.2 Story prompt and preference fields

Figure 8.3 Generated comic preview page

Figure 8.4 Comic panel with narration and dialogue

Figure 8.5 Local application running in development environment

Figure 8.6 Additional comic preview result

Figure 8.7 Panel result and image display area

Chapter 1 – Introduction

1.1 Project Description

ComicCraft is a web-based application that uses Generative AI to create personalized comic book stories and illustrations from user-provided prompts. It is built with FastAPI and integrates Google Gemini models for story generation with Stable Diffusion for comic-style image generation. The application automates storylines, dialogues, scene descriptions, and visual illustrations.

The user enters a story prompt and selects or provides a main character name, setting, story tone, and preferred art style. The system processes these inputs and generates a panel-by-panel comic with corresponding illustrations. The result is presented through a browser interface and can be exported as a PDF containing images and narrative content.

1.2 Problem Statement

Traditional comic creation combines several tasks: developing a plot, writing dialogue, illustrating scenes, arranging panels, and preparing a final document. These tasks may be challenging for casual users who do not have drawing or design experience. The project addresses this problem by integrating text generation and image generation into a single workflow.

1.3 Objectives

- Generate structured comic outlines from user prompts.
- Create narration and character dialogue automatically.
- Generate comic-style illustrations for individual panels.
- Provide a simple web interface for entering story preferences.
- Display generated comic content sequentially for review.
- Compile the generated comic into a downloadable PDF.
- Make comic creation accessible to non-artists and casual users.

Chapter 1 – Introduction (Continued)

Project Scope

The scope described in the project documentation is single-comic generation from a prompt through a local web application. It includes input collection, outline generation, story expansion, panel illustration, preview, and PDF export. The supplied documentation does not describe a production account system or a persistent personal comic library.

Purpose and Significance

The project demonstrates how a user-friendly web interface can connect language models and image-generation software. Instead of requiring the user to switch between separate writing, illustration, and layout tools, ComicCraft organizes these functions behind one application workflow.

Intended Users

- Casual users who want to develop a short visual story.
- Students exploring web development and Generative AI integration.
- Storytellers and hobbyists who need a draft comic concept.
- Users who want to preview and export a short comic in PDF form.

Chapter 2 – Generative AI Platform Overview

2.1 Introduction to Generative AI

Generative AI refers to artificial intelligence systems that create new content from instructions or prompts. In ComicCraft, the content includes structured story text, narration, dialogue, and comic illustrations.

The supplied project design uses Gemini Flash to create a structured comic outline, Gemini Pro to expand the outline into narration and character dialogue, and Stable Diffusion to generate illustrations. These components form a pipeline in which the output from one stage becomes input or context for the next stage.

Text Generation in ComicCraft

The text-generation process begins with user preferences. The outline stage organizes the story into five panels. The detailed story stage then adds narration and dialogue to the outline. This separation makes it possible to organize the overall sequence before expanding individual scenes.

Image Generation in ComicCraft

Stable Diffusion is integrated through Hugging Face Diffusers. The image-generation component receives scene descriptions or prompts and produces panel illustrations. The generated image files are then associated with the corresponding panel in the preview and export layout.

Chapter 2 – Platform Overview (Continued)

AI Generation Pipeline

1. Collect the user's story prompt and preferences.
2. Use Gemini Flash to generate a structured five-panel outline.
3. Use Gemini Pro to expand the outline into narration and dialogue.
4. Use Stable Diffusion to generate an illustration for each panel.
5. Match each illustration with its panel text and assemble the preview.
6. Use the export component to create the PDF.

Role of the Web Backend

FastAPI acts as the coordinating backend. It receives form or JSON input, validates the request, invokes the relevant generation functions, and returns the generated comic data to the frontend. It also exposes routes used for generation, image testing, and export confirmation.

Benefits and Boundaries

The documented benefit is the integration of multiple creative steps into one workflow. Generated content may vary between runs, and image generation depends on the configured libraries, model availability, hardware, and credentials. The supplied project documentation does not provide measured generation-time or accuracy benchmarks.

Chapter 3 – Project Analysis

3.1 Existing Behaviour

In a conventional workflow, a creator independently develops a story idea, outlines scenes, writes narration and dialogue, creates illustrations, organizes panels, and prepares the final document. These activities are commonly performed through separate creative and technical steps.

3.2 Problems in Existing Behaviour

- Comic creation may require writing, illustration, and layout skills.
- Creating multiple consistent panels manually can take significant effort.
- Non-artists may find visual comic creation difficult.
- Narration, dialogue, images, and PDF layout must be coordinated.
- Changing tone or art style can require repeating creative steps.

3.3 Proposed Solution

ComicCraft proposes an integrated AI-assisted workflow. The user enters a prompt and preferences. FastAPI coordinates the AI pipeline; Gemini Flash creates the outline, Gemini Pro generates detailed narration and dialogue, Stable Diffusion generates illustrations, and the layout/export components assemble the comic.

3.4 Project Scenarios / Use Cases

- Personalized comic: enter a story idea such as a brave fox exploring an enchanted forest and receive a multi-panel story.
- Style and tone iteration: choose a funny tone and comic-book art style, then generate content aligned with those preferences.
- PDF export: review the generated comic and select the download action to create a PDF and show an export confirmation.

Chapter 3 – Project Analysis (Continued)

Use Case: Create a Comic

Actor: User. Precondition: the local application is running and required AI services are configured.

Main flow: open the home page, enter a prompt, select character and style options, submit the form, wait for generation, and review the panel preview. The user may then export the result.

Use Case: Test Image Generation

Actor: Developer or tester. The documented `/test-image` route is used to check the image-generation component separately. This helps isolate image-generation configuration from the full story workflow.

Use Case: Generate Through API

The project includes a JSON-based generation route at `/generate-comic/json`. It accepts structured request data and connects the API request to the comic-generation workflow. FastAPI's interactive documentation at `/docs` can be used to inspect and exercise available API operations.

System Inputs and Outputs

Inputs include the story prompt, main character name, setting, story tone, and art style. Outputs include a structured five-panel outline, panel titles or descriptions, narration, dialogue, illustrations, a browser preview, and a downloadable PDF.

Chapter 4 – System Architecture and Design

4.1 Architecture Overview

ComicCraft follows a three-part architecture: frontend, backend, and AI integration layer. An export component produces the final PDF.

Frontend Layer

The frontend uses HTML, CSS, and Jinja2 templates. The `index.html` page collects user input. The `comic_preview.html` page displays generated panels and text. The `export_success.html` page confirms completion of the export action.

Backend Layer

FastAPI manages routes, request validation, AI workflow coordination, layout construction, and PDF export. Pydantic is used for structured request data. Error handling includes `HTTPException` and `try-except` blocks, as described in the project documentation.

AI and Export Layers

The AI layer integrates Gemini APIs and Hugging Face Diffusers with Stable Diffusion. The export layer uses FPDF to compile panel images and narrative content into a multi-page PDF.

Chapter 4 – System Architecture and Design (Continued)

Architecture Component Summary

Component	Technology / Module	Responsibility
Frontend	HTML, CSS, Jinja2	Collect preferences and display output
Backend	FastAPI, routes.py	Routes, validation, orchestration
Outline generation	gemini_flash.py	Structured five-panel outline
Story generation	gemini_pro.py	Narration and dialogue
Image generation	image_generator.py	Stable Diffusion panel illustrations
Layout	layout_builder.py	Pair text and images
PDF export	exporters.py / FPDF	Compile and export comic PDF

4.2 Project Workflow

7. User enters story prompt and preferences.
8. FastAPI receives and validates the input.
9. Gemini Flash generates a structured five-panel outline.
10. Gemini Pro expands the outline into narration and dialogue.
11. Stable Diffusion generates an illustration for each panel.
12. The layout builder matches images with story content.
13. The frontend displays the completed comic panel by panel.
14. The export module creates a downloadable PDF.
15. The user receives an export-success confirmation.

Chapter 5 – Requirements and Environment Setup

5.1 Hardware Requirements

- A computer capable of running Python and the FastAPI development server.
- Sufficient storage for source files, generated images, model assets, and exported PDFs.
- Internet connectivity for configured AI services and APIs.
- Image-generation performance depends on the local or configured compute environment. The supplied source does not specify a minimum CPU, RAM, or GPU specification.

5.2 Software Requirements

- Python and pip.
- FastAPI and Uvicorn ASGI server.
- HTML, CSS, and Jinja2 templates.
- Google Gemini API integration.
- Hugging Face Diffusers and Stable Diffusion.
- PyTorch and supporting image-generation libraries.
- FPDF for PDF generation.
- Git for version control.
- Python virtual environment such as venv, virtualenv, or conda.

5.3 ServiceNow Instance Requirements

The supplied ComicCraft project documentation does not specify a ServiceNow instance, ServiceNow application, ServiceNow API, or ServiceNow-specific configuration. Therefore, ServiceNow instance requirements are not defined for the documented project. This section is retained because it appears in the requested report outline; no ServiceNow configuration has been assumed.

Chapter 5 – Environment Setup (Continued)

Local Environment Setup

The project documentation describes using a Python virtual environment and installing dependencies from the project's requirements file. The commands below are included in the supplied appendix.

```
python -m venv comiccraft-env  
comiccraft-env\Scripts\activate (Windows)  
pip install fastapi uvicorn jinja2 python-multipart google-generativeai  
diffusers transformers fpdf Pillow accelerate  
uvicorn app.main:app --reload
```

Configuration and Credentials

Sensitive service credentials, including the Gemini API key and Hugging Face API token, are configured through environment variables according to the project documentation. Credentials should not be placed directly in source files or included in submitted screenshots.

Local Access

The documented local homepage is <http://127.0.0.1:8000>. FastAPI's interactive API documentation is available at <http://127.0.0.1:8000/docs> when the server is running.

Chapter 6 – Project Implementation

6.1 Milestone 1 – Model Selection and Architecture

The first milestone covers selecting the AI models and defining the application architecture. Gemini Flash is assigned structured comic-outline generation, Gemini Pro is assigned detailed narration and dialogue generation, and Stable Diffusion is assigned comic-style image generation.

The development environment is prepared using Python, a virtual environment, required packages, and Uvicorn. The architecture separates user interface, backend orchestration, AI services, layout construction, and export.

Milestone 1 Deliverables

- Defined model responsibilities.
- Defined frontend, backend, AI, and export layers.
- Prepared the Python development environment.

Chapter 6 – Project Implementation (Continued)

6.2 Milestone 2 – Core Functionalities Development

Core functions support outline generation, story generation, image generation, layout construction, and PDF export.

Function	Purpose
<code>generate_outline()</code>	Creates a five-panel outline.
<code>generate_story()</code>	Produces narration and dialogue.
<code>generate_image()</code>	Creates and saves panel illustrations.
<code>build_comic_layout()</code>	Organizes panel content and images.
<code>save_pdf()</code>	Compiles the final comic into PDF.

6.3 Milestone 3 – routes.py Development

The routes.py module connects user inputs with the comic-generation workflow. Documented routes include the homepage, comic generation, JSON-based generation, export-success confirmation, and image testing. Form and JSON inputs are processed using FastAPI and Pydantic.

Chapter 6 – Project Implementation (Continued)

6.4 Milestone 4 – Frontend Development

The frontend uses HTML, CSS, and Jinja2. `index.html` collects the prompt and preferences; `comic_preview.html` displays generated panels; `export_success.html` confirms the export. Input fields include story prompt, character name, setting, tone, and art style.

6.5 Milestone 5 – Deployment

The application is prepared for local deployment using a Python virtual environment and `requirements.txt`. Environment variables hold sensitive API information. Uvicorn launches the FastAPI application.

6.6 Milestone 6 – Local Verification and End-to-End Flow

Local verification uses the FastAPI development server and browser-based testing. The end-to-end path covers prompt entry, AI text generation, image creation, comic preview, and PDF export. The `/docs` endpoint provides an interactive API testing interface.

Chapter 7 – Testing and Validation

Testing and validation are performed by running the application locally and checking the generation workflow. The project documentation identifies /docs as an API testing interface and /test-image as a route for checking image generation independently.

Functional Validation Areas

- Homepage loading and input collection.
- Form-based generation through /generate.
- JSON-based generation through /generate-comic/json.
- Independent image generation through /test-image.
- Panel-by-panel preview rendering.
- PDF generation and export.
- Export-success page display.
- Error handling for API or input-processing failures.

Validation Approach

Each function can be exercised separately before testing the complete flow. For example, image generation can be checked through the dedicated test route, while the complete workflow verifies that generated text and images are associated with the correct panels.

Testing Record Template

Test ID	Test activity	Expected observation	Status / date
TC-01	Open home page	Input form loads	To be recorded
TC-02	Submit story prompt	Comic generation starts	To be recorded
TC-03	Open preview	Panels and text display	To be recorded
TC-04	Generate PDF	PDF export completes	To be recorded
TC-05	Open /docs	API documentation loads	To be recorded

Chapter 7 – Testing and Validation (Continued)

End-to-End Validation

An end-to-end validation session begins with a user prompt and selected preferences. The tester observes whether the backend accepts the input, whether text generation returns structured content, whether images are generated and saved, whether the preview associates content panel by panel, and whether the PDF export action reaches the confirmation page.

Limitations of the Evidence

The screenshots supplied with this report show the browser interface, preview output, and local development environment. They do not establish numerical performance benchmarks, load-testing results, or formal usability-study outcomes. Such measurements should be recorded only if separately conducted.

Acceptance Criteria

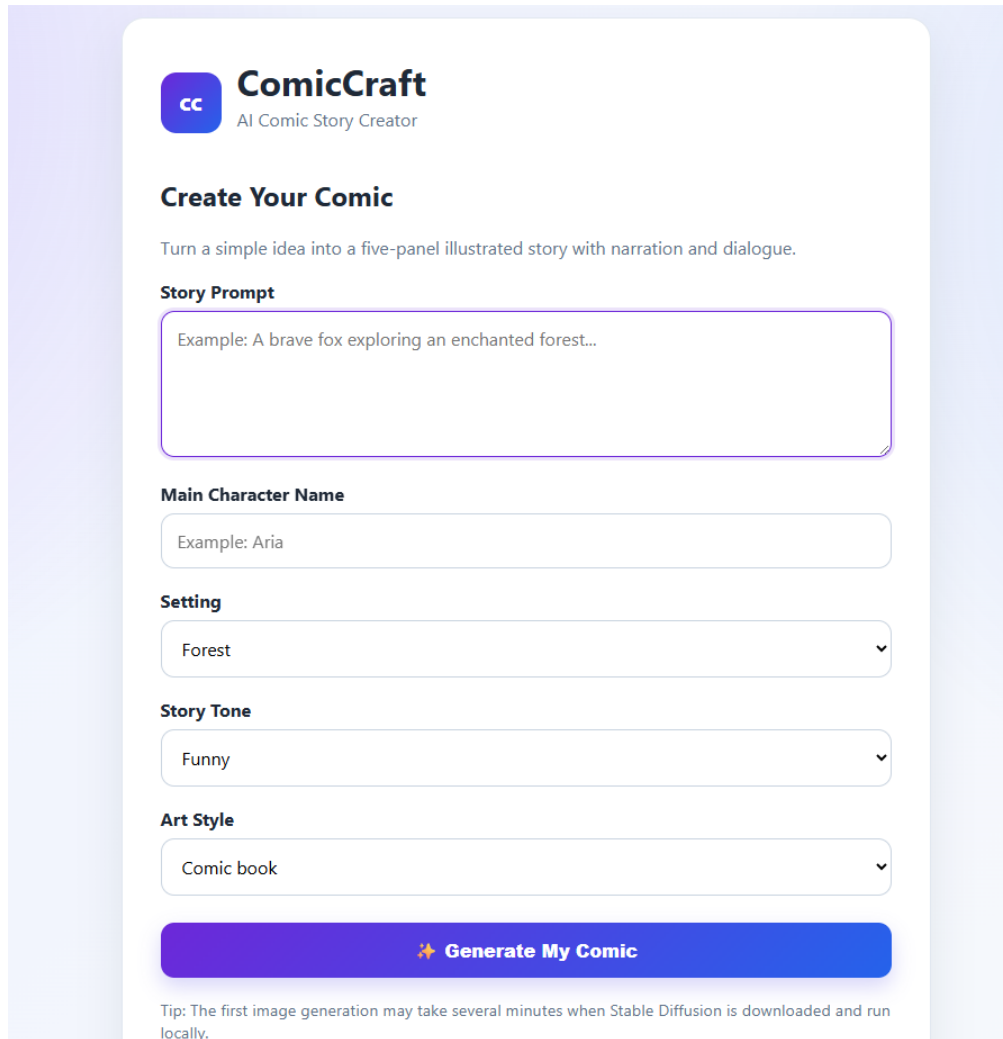
- The home page presents the required input controls.
- The comic preview presents multiple panels with text content.
- The image area is associated with the relevant panel.
- The PDF export action provides a downloadable document or visible result.
- The documented local routes can be accessed while the server is running.

Chapter 8 – Results and Discussion

The supplied project screenshots show the ComicCraft input form, a local browser session, generated comic-preview cards, panel descriptions, captions, narration, dialogue, and image display areas. The documented output is a five-panel comic generated from the user's story information.

8.1 Home Page and Input Form

The home page identifies the application as ComicCraft – AI Comic Story Creator. It provides a story prompt field, main character name field, setting selector, story-tone selector, art-style selector, and a Generate My Comic button.



The screenshot displays the ComicCraft web interface. At the top left is the 'ComicCraft' logo with the tagline 'AI Comic Story Creator'. Below this is the heading 'Create Your Comic' followed by a descriptive sentence: 'Turn a simple idea into a five-panel illustrated story with narration and dialogue.' The form consists of several input fields: a large text area for the 'Story Prompt' (with an example: 'A brave fox exploring an enchanted forest...'), a text field for the 'Main Character Name' (with an example: 'Aria'), and three dropdown menus for 'Setting' (set to 'Forest'), 'Story Tone' (set to 'Funny'), and 'Art Style' (set to 'Comic book'). A prominent blue button labeled 'Generate My Comic' with a magic wand icon is positioned below the dropdowns. At the bottom, a small tip states: 'Tip: The first image generation may take several minutes when Stable Diffusion is downloaded and run locally.'

Figure 8.1 – ComicCraft home page and story input form

Chapter 8 – Results and Discussion (Continued)

8.2 Prompt and Preference Entry

The form allows a user to describe a story idea and set creative preferences before generation. The supplied interface screenshot shows Forest, Funny, and Comic book as selected example values.

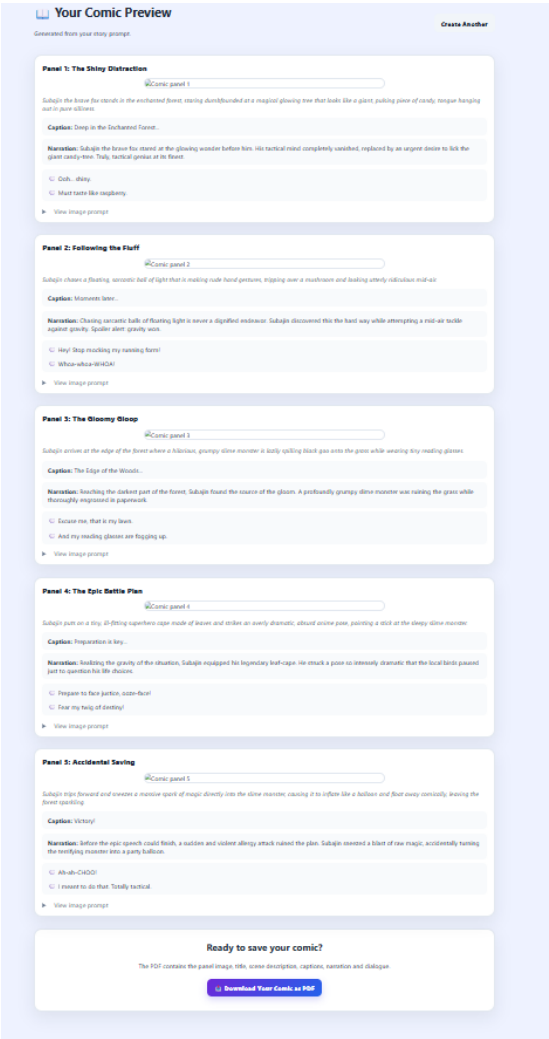


Figure 8.2 – Application interface shown during local development

Chapter 8 – Results and Discussion (Continued)

8.3 Generated Comic Preview

The preview organizes the generated story into numbered panels. Each panel contains a title, illustration area, scene description, caption, narration, and dialogue where provided.

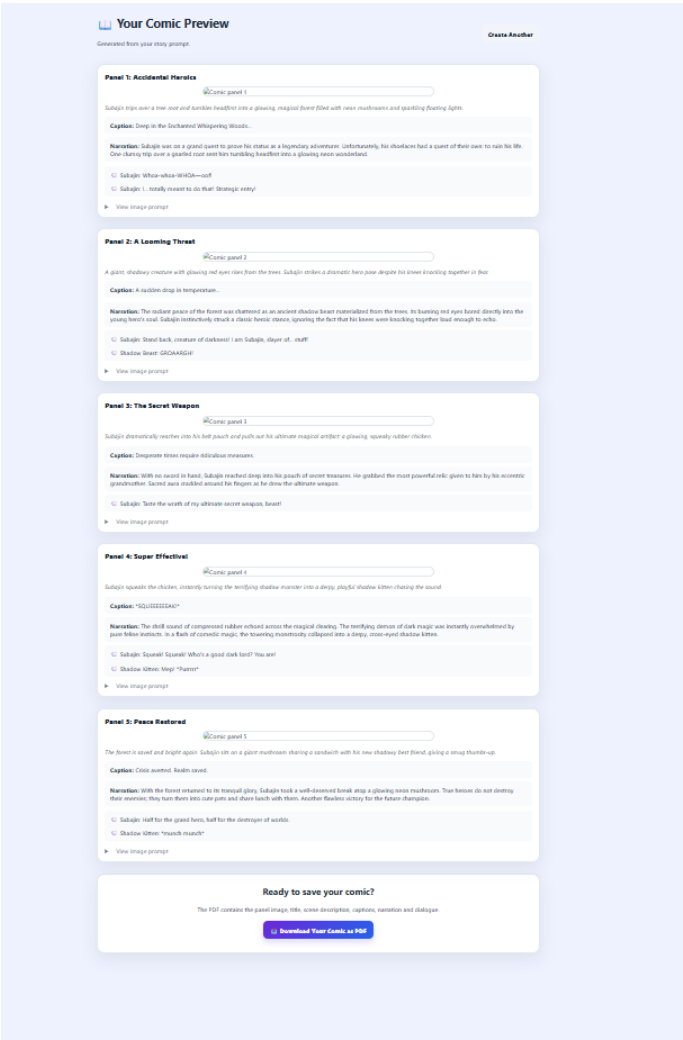


Figure 8.3 – Generated comic preview with panel cards

Chapter 8 – Results and Discussion (Continued)

8.4 Panel Content and Narration

The supplied preview shows an example comic sequence featuring a character named Subaj. The visible panels include narrative descriptions and dialogue, illustrating how story text is organized for review.

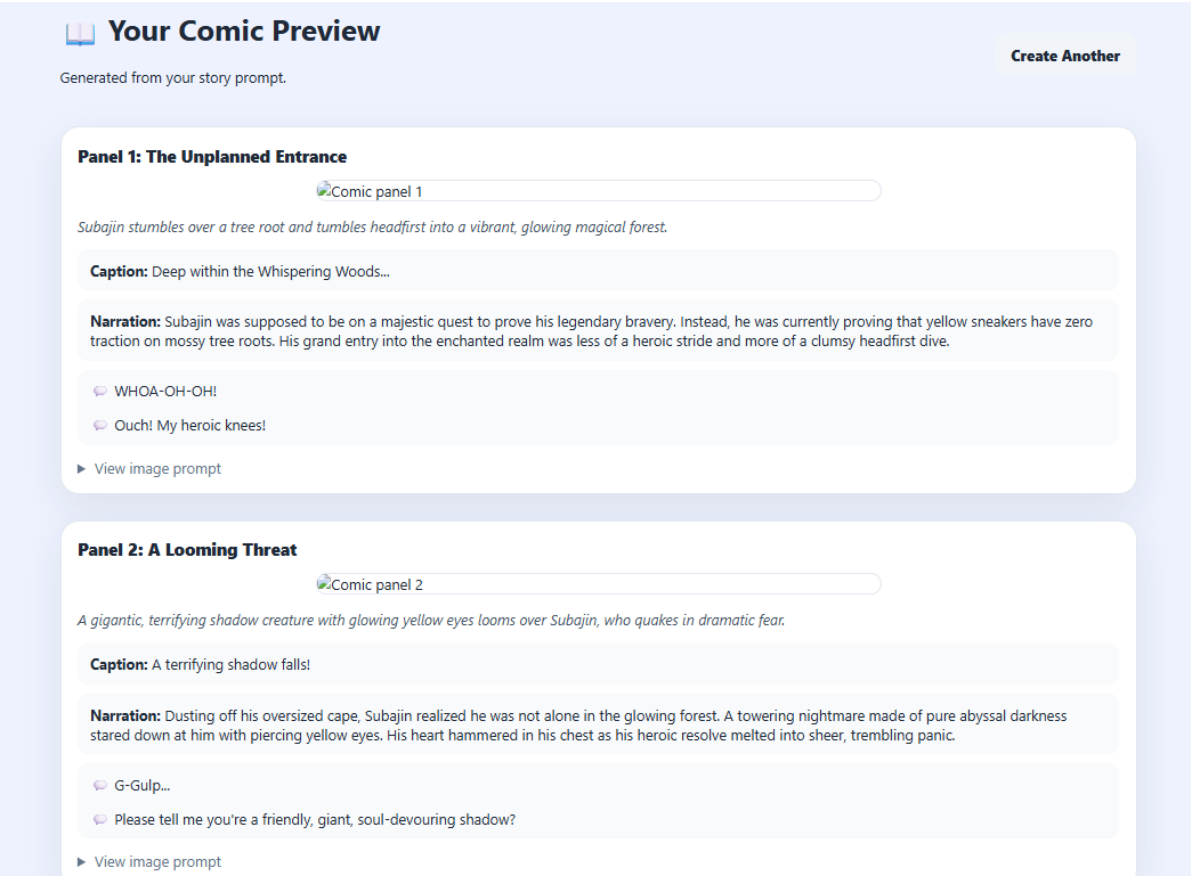


Figure 8.4 – Individual panel with scene description, caption, narration, and dialogue

Chapter 8 – Results and Discussion (Continued)

8.5 Local Application and Development Environment

The development screenshot shows the application running at the local address 127.0.0.1:8000 within a browser, with the project files and Uvicorn terminal visible in the development environment.

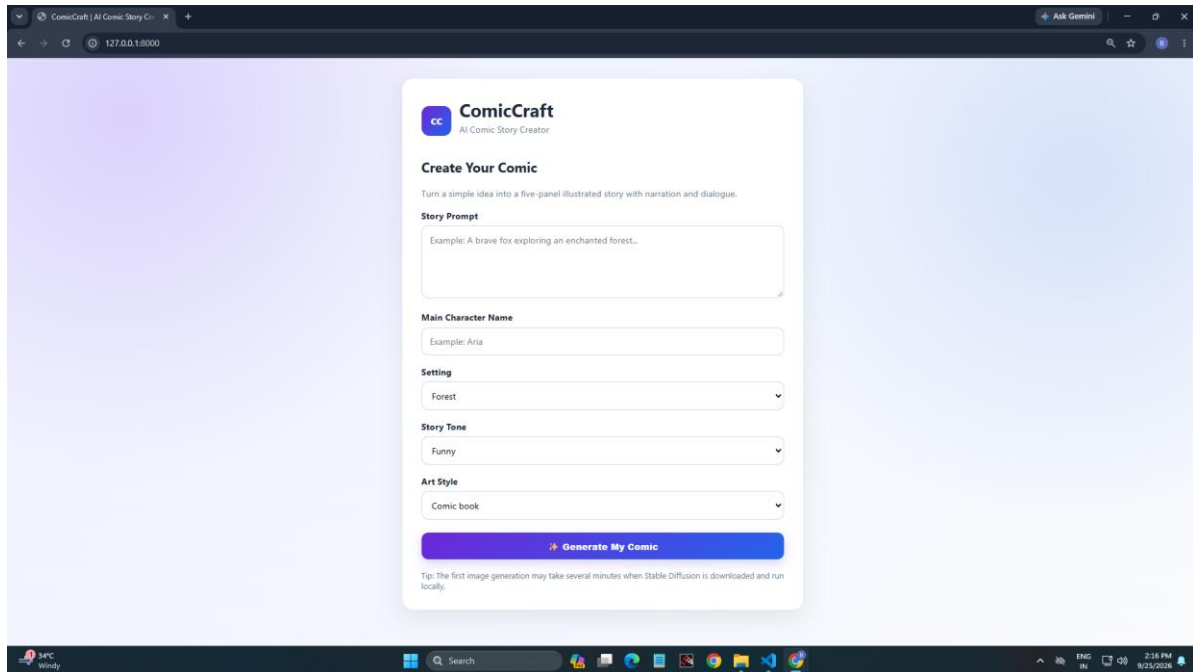


Figure 8.5 – Local ComicCraft application and development server

Chapter 8 – Results and Discussion (Continued)

8.6 Additional Generated Output

Additional screenshots show generated comic preview content with panel headings, captions, narration, and dialogue. The examples demonstrate that the preview is arranged as a sequential comic rather than a single block of text.

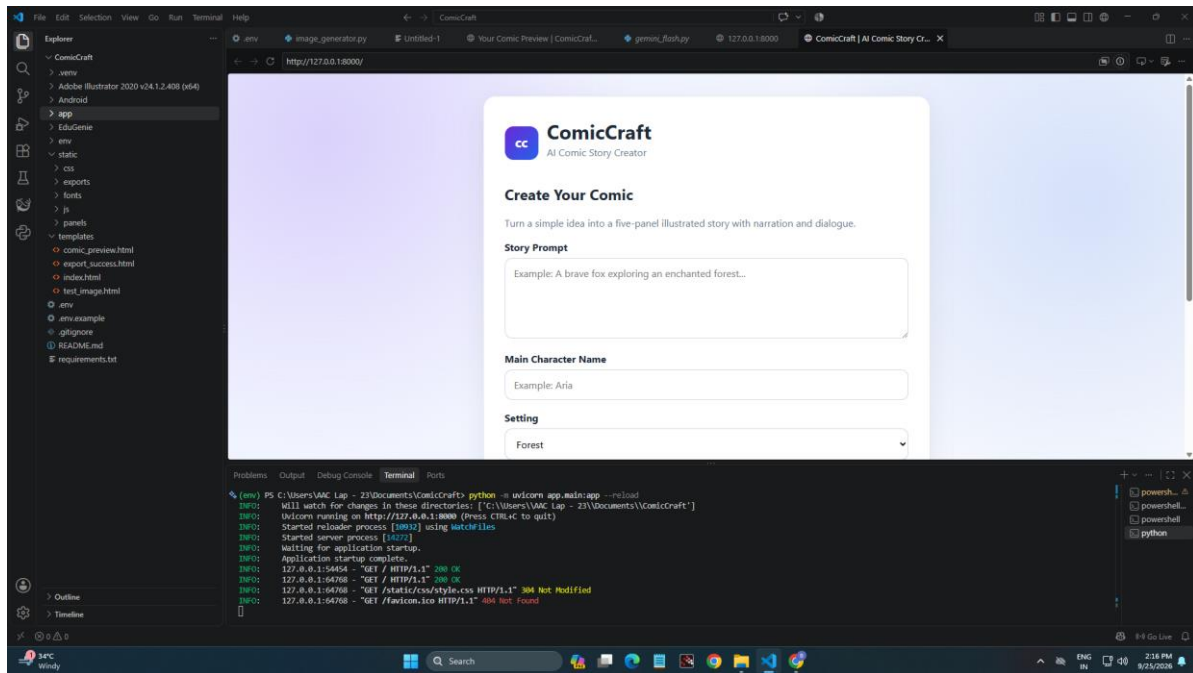


Figure 8.6 – Additional generated comic preview

Chapter 8 – Results and Discussion (Continued)

8.7 Panel Preview and Image Area

The final supplied screenshot displays a panel result and its image area. In the screenshot, the image element displays alternative text rather than a rendered illustration; this is recorded as an observed display issue in the supplied evidence and should be rechecked during testing.

Panel 1: The Discovery

 Comic panel 1

Arjun, a shy college student with messy dark hair and glasses, sits alone in a quiet corner of the old campus library. He opens a dusty, leather-bound textbook and finds a mysterious, intricate glowing silver watch resting inside, emitting a soft blue light that illuminates his surprised face.

Caption: THE OLD CAMPUS LIBRARY

Narration: Arjun always preferred the quietest corners of the library to hide from the noise of college life. But when he opens an ancient, forgotten textbook, a strange artifact greets him with an impossible, humming glow. The intricate silver watch seems to wait specifically for him.

Figure 8.7 – Panel result showing the image display area

Discussion

The screenshots support the presence of a working input page, a generated preview layout, and local development access. They also show that some image areas may not have loaded in the captured browser output. Therefore, image rendering should be verified independently before final demonstration.

Chapter 9 – Challenges and Troubleshooting

The application combines FastAPI, Gemini models, Stable Diffusion, Jinja2, FPDF, environment variables, and local deployment. Each integration must be configured and tested.

Common Challenges

- API configuration: Gemini and Hugging Face credentials must be configured through environment variables.
- Dependency management: required Python libraries must be installed in the active virtual environment.
- AI generation coordination: text and image generation must occur in the correct sequence.
- Image handling: generated files must be saved and referenced correctly by the layout and frontend.
- Template rendering: backend data must be passed to Jinja2 templates using the expected names.
- PDF export: images and narrative text must be organized before FPDF creates the file.
- Local server testing: Uvicorn must run successfully and expected routes must be reachable.

Chapter 9 – Challenges and Troubleshooting (Continued)

Troubleshooting Checklist

16. Confirm the virtual environment is activated before installing or running packages.
17. Check that required packages are installed in the same Python environment used by Uvicorn.
18. Confirm API credentials are present in the environment and are not exposed in source control.
19. Check the terminal output for the route or service that reports an error.
20. Verify that generated image paths point to existing files served by the application.
21. Check the browser developer console and server logs when an image is missing.
22. Test the export route after confirming the comic preview contains the expected data.

Image Display Issue Observed

Some supplied preview screenshots show a broken image placeholder with alternative text such as “Comic panel 1.” This report does not assume the cause. Possible checks within the documented workflow include confirming that the image generator saved the file, that the layout references the correct filename, and that the frontend can access the image URL.

Chapter 10 – Future Enhancements

The project documentation identifies the following possible future enhancements.

- User profiles and account management.
- A personal comic library for saving previously generated comics.
- Support for multi-page story arcs.
- Additional customization options for characters and visual styles.
- Potential cloud deployment after successful local verification.
- Further improvements to the user interface and comic-generation workflow.

Possible Extension Areas

A future version could expand the current single-comic workflow into a library of stories, allow users to revisit earlier outputs, and provide additional controls over character appearance and visual consistency. Cloud deployment is a potential later step after local verification.

Enhancement Planning Considerations

Future work should be planned around user needs, technical feasibility, model and service requirements, storage, and testing. These items are possibilities described by the project documentation, not claims that the features are already implemented.

Chapter 11 – Conclusion

Project Summary

ComicCraft demonstrates an end-to-end Generative AI application for creating personalized comic stories and illustrations. It combines FastAPI, Google Gemini models, Stable Diffusion, Jinja2 templates, and FPDF to transform user prompts into structured comic panels, narration, generated images, and a downloadable PDF.

Objectives Achieved

- A web-based interface was designed to collect comic-generation preferences.
- AI-based structured comic-outline generation was incorporated.
- AI-based narration and dialogue generation was incorporated.
- Comic-style image generation was integrated into the workflow.
- Generated content was organized into panel layouts.
- A PDF export workflow was included.
- The process was connected through a FastAPI backend.

Learning Outcomes

- Understanding of Generative AI model integration.
- Practical use of FastAPI for backend development and API routing.
- Experience with Jinja2 dynamic web templates.
- Understanding of AI image generation using Diffusers and Stable Diffusion.
- Experience with Python environments and dependency management.
- Understanding of PDF generation and content layout.
- Experience connecting frontend, backend, AI services, and export functionality.

References

FastAPI Documentation – <https://fastapi.tiangolo.com/>

Uvicorn Documentation – <https://www.uvicorn.org/>

Python Official Documentation – <https://docs.python.org/3/>

Jinja Documentation – <https://jinja.palletsprojects.com/>

Google Gemini API Documentation – <https://ai.google.dev/>

Hugging Face Diffusers Documentation – <https://huggingface.co/docs/diffusers/>

PyTorch Documentation – <https://pytorch.org/docs/>

FPDF Documentation – <https://pyfpdf.github.io/fpdf2/>

Git Documentation – <https://git-scm.com/doc>

Python venv Documentation – <https://docs.python.org/3/library/venv.html>

Note: These references correspond to the technologies identified in the supplied ComicCraft project documentation. Consult the official documentation for current installation instructions and API details.

Appendix A – Key Development Commands

A.1 Environment and Run Commands

```
python -m venv comiccraft-env
```

```
comiccraft-env\Scripts\activate # Windows
```

```
pip install fastapi uvicorn jinja2 python-multipart google-generativeai  
diffusers transformers fpdf Pillow accelerate
```

```
uvicorn app.main:app --reload
```

A.2 Local Access

Application: <http://127.0.0.1:8000>

FastAPI API documentation: <http://127.0.0.1:8000/docs>

Appendix B – Main Application Components

File / module	Documented responsibility
gemini_flash.py	Structured comic outline generation
gemini_pro.py	Narration and dialogue generation
image_generator.py	Stable Diffusion image generation
layout_builder.py	Comic panel layout construction
exporters.py	PDF generation
routes.py	FastAPI route handling
index.html	User input page
comic_preview.html	Generated comic preview
export_success.html	Export confirmation

Appendix C – Screenshot and Demonstration Notes

The screenshots included in Chapter 8 were supplied with the project request. Captions describe the visible content of each screenshot. Before final submission, the student should confirm that each image is legible in the printed or exported report and replace any screenshot if a newer verified result is available.

Suggested demonstration sequence:

23. Start the virtual environment and run the FastAPI application.
24. Open the local home page and enter a sample prompt.
25. Select character, setting, tone, and art style.
26. Generate and review the comic preview.
27. Check that panel images load correctly.
28. Download the comic PDF and show the export result.
29. Open /docs to demonstrate the API documentation.